

TP N° 3

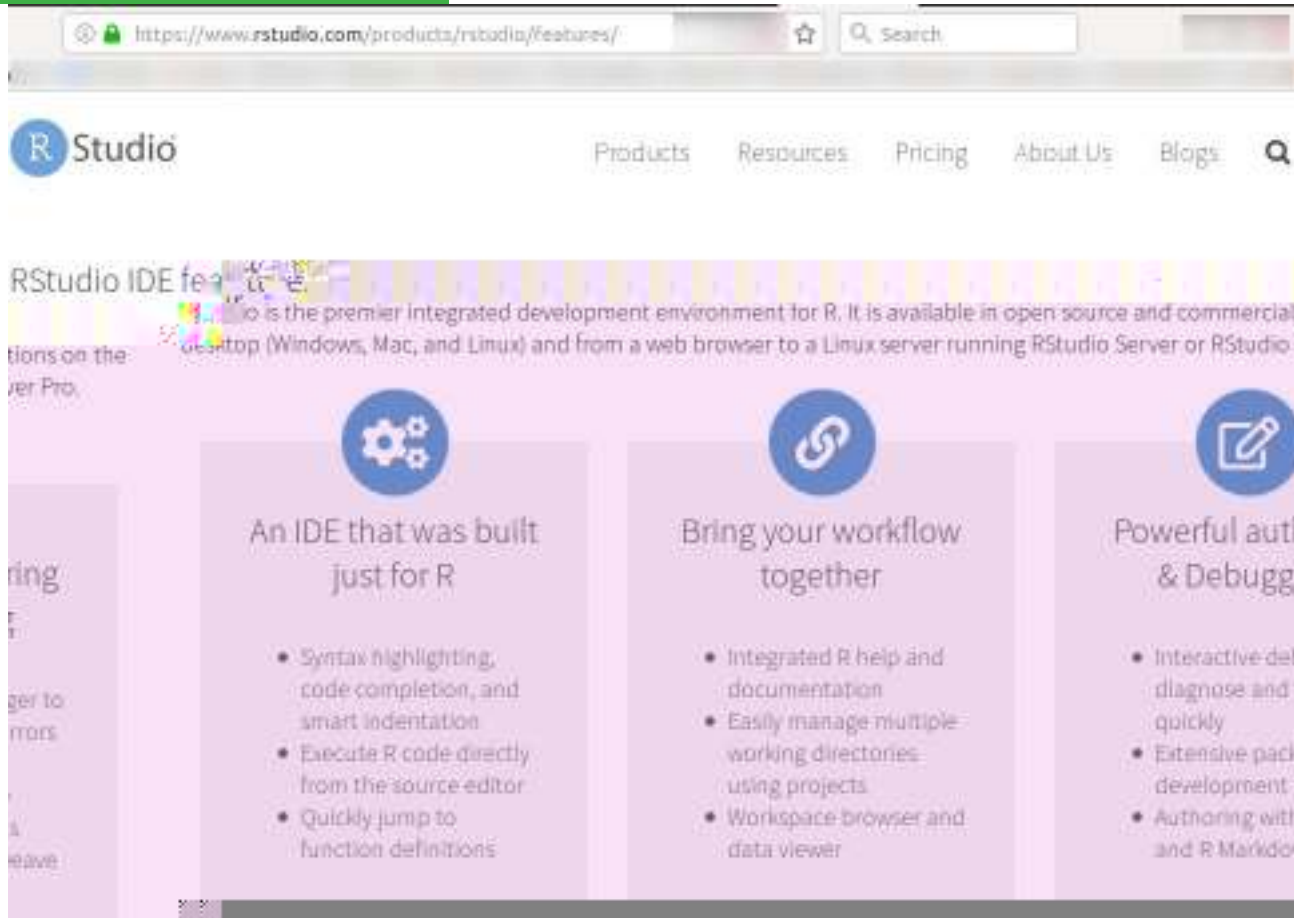
Rstudio, Facteurs, Listes & Fonctions-utilisateur

But du TP :

Utiliser Rstudio, manip de graphes basique, facteurs et listes.

Énoncé :

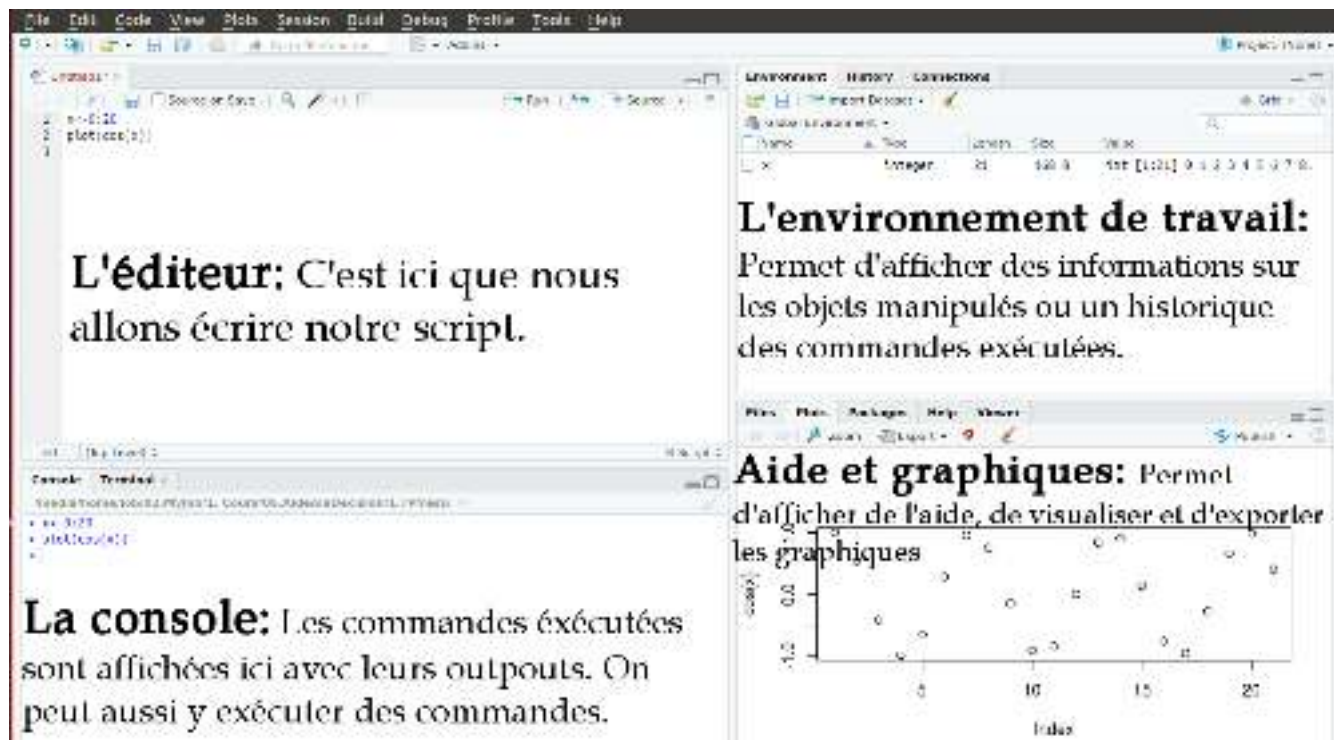
A. Présentation de RStudio



Rstudio est un IDE multi-plateforme téléchargeable depuis le lien
<https://www.rstudio.com/products/rstudio/download/>

Après installation et lancement, une fenêtre découpée en quatre zones s'affiche :

1. **La zone d'édition** : permet l'édition de fichiers source R (avec coloration syntaxique et auto-complétion des noms de fonction et d'objet avec la touche **tabulation**). L'utilisation du raccourci **Ctrl+Entrée** permet d'exécuter directement une ligne ou une sélection du fichier source.
2. **La zone console** : Affiche les commandes exécutées et leurs outputs.
 - Saisir le code : `x<-1:20 ; x<-x+1` ; dans la zone d'édition, puis le sélectionner et l'exécuter avec **Ctrl+Entrée** plusieurs fois. Qu'est-ce que vous remarquez.



Ces deux zones procurent donc deux manières d'interagir avec R: en utilisant la console, on peut exécuter des commandes pour vérifier leur effet mais sans les sauvegarder. On utilisera l'éditeur pour les sauvegarder nos commandes dans des fichiers de scripts.

3. **L'environnement de travail** : on peut l'utiliser pour avoir des informations sur les objets manipulés.
 - Exécuter le code : `mat<-matrix(1:100, nrow=20,byrow=T)` puis visualiser le contenu de `mat`.
4. **La zone d'aide et graphiques** : Permet d'afficher l'aide (onglet help), de visualiser un graphe (onglet plots), de visualiser les packages installés ou d'en installer de nouveaux (onglet packages) et de naviguer dans le système de fichiers (onglet file) pour par exemple créer un répertoire et changer le répertoire de travail.
 - Exécuter le code `x<-0:20 ; plot(cos(x))`. Qu'elle est la méthode d'affichage par défaut ?
 - Exécuter `?plot` puis déduire l'argument et la valeur qui permet d'avoir une courbe continue.
 - En utilisant le menu Export sauvegarder le graphique obtenu avec le format PDF.
 - Utiliser l'onglet Files pour ouvrir votre fichier PDF.

B. Facteurs

Les facteurs sont une structure de données pour représenter des données **qualitatives** (ou catégorielles). Un facteur est composé d'une suite de valeurs catégorielles et d'un tableau **levels** contenant ces valeurs catégorielles triées sans répétition.

Exemple : Voici un code et l'output associé:

```
mentions <- factor(c("Tres Bien","Bien","Assez Bien","Passable","Faible"))
mentions
```

```
[1] Tres Bien  Bien      Assez Bien Passable  Faible
Levels: Assez Bien Bien Faible Passable Tres Bien
```

Par défaut, **factor()** construit l'attribut *levels* en créant un vecteur ordonné des valeurs du facteur. Si, nous souhaitons préciser que les mentions peuvent avoir d'autres valeurs (excellent et tres faible par exemple) il suffit de spécifier l'univers des valeurs par l'argument *levels* comme suit :

```
mentions <- factor(c("Tres Bien","Bien","Assez Bien","Passable","Faible"),
```

```

levels=c("Excellent","Tres Bien","Bien","Assez Bien","Passable",
         "Faible","tres faible")
)#fin de création de facteur

```

Il est possible d'assigner des noms aux éléments d'un facteur avec la fonction **names()**. Par exemple, on peut ajouter au facteur **mentions** des noms pour distinguer et numéroter les devoirs maison (DM) et les devoirs surveillés (DS) :

```

mentions <- factor(c("Tres Bien","Bien","Assez Bien","Passable","Faible"),
                  levels=c("Excellent","Tres Bien","Bien","Assez Bien","Passable",
                           "Faible","tres faible"))
names(mentions)=c("DM1","DS1","DS2","DM2","DS3")
mentions
> mentions
      DM1      DS1      DS2      DM2      DS3
Tres Bien  Bien Assez Bien  Passable  Faible
Levels: Excellent Tres Bien Bien Assez Bien Passable Faible tres faible

```

- Est-ce-qu'il y a une différence concernant l'attribut **levels** entre sa création implicite et explicite ?

La taille d'un facteur (ou d'un vecteur) est donnée par la fonction **length()**. On peut modifier la taille en lui affectant une nouvelle valeur. Exécuter les commandes suivantes et commenter :

```

length(mentions)
mentions
length(mentions) <- 3
mentions
length(mentions) <- 10
mentions

```

Comme pour les vecteurs, on peut accéder aux éléments du facteur en spécifiant leurs indices numériques ou leurs noms entre crochets. Par exemple, exécuter les commandes suivantes :

```

mentions[3]           #la mentions du 3ème devoir de l'année.
mentions[c("DS2","DS4")] #les mentions des devoirs surveillés (DS) numéro 2 et 4.
mentions[1:4]          #les quatre premières mentions de l'année.

```

Pour extraire l'attribut *niveaux* d'un facteur on utilise la fonction **levels(<identiffacteur>)**. Cette même fonction permet de mettre à jour les niveaux. Par exemple, exécuter les suivantes :

```

levels(mentions)
levels(mentions) <- c(levels(mentions), "Mediocre")

```

C. Listes

Une **liste** est une structure de données pouvant contenir des données de différents types. Une liste est créée en utilisant la fonction **list()**. Exécuter chacune des commandes suivantes, afficher et commenter :

```

> maListe <- list("a" = 2.75, "b" = TRUE, "c" = 1:5)
> maListe
$a
[1] 2.75
$b
[1] TRUE
$c
[1] 1 2 3 4 5
> str(maListe)
list of 3
 $ a: num 2.75
 $ b: logi TRUE
 $ c: int [1:5] 1 2 3 4 5
> maListe$a

```

```

[1] 2.75
> maListe <- list(2.75, TRUE, 1:5)
> maListe
[[1]]
[1] 2.75
[[2]]
[1] TRUE
[[3]]
[1] 1 2 3 4 5
> str(maListe)
list of 3
 $ : num 2.75
 $ : logi TRUE
 $ : int [1:5] 1 2 3 4 5
> maListe[[1]]
[1] 2.75
> maListe[[3]][3]
[1] 3

```

En utilisant des listes, nous pouvons attribuer des noms de lignes et de colonnes à une matrice en utilisant la fonction `dimnames()`. Nous pourrions ainsi accéder à ses éléments par nom.

```

> A<-matrix(c(1,2,3,4,5,6),nrow=2)
> dimnames(A) <- list(c("row1", "row2"),c("col1", "col2", "col3"))
> A
      col1 col2 col3
row1    1    3    5
row2    2    4    6
> A["row2", "col3"] # element at 2nd row, 3rd column
[1] 6
> A["row1",]
col1 col2 col3
  1    3    5

```

`as.list()` peut être utilisé pour convertir un vecteur en une liste. Chaque élément du vecteur sera un élément à part de la liste.

```

> fruits_vector <- c("apple", "banana", "orange")
> fruits_list <- as.list(fruits_vector)
> fruits_list

```

`append()` ajoute un élément à une liste et retourne la liste modifiée.

```

> fruits_list <- append(fruits_list, "grape")

```